Ecole Nationale de l'Aviation Civile
7 Av. Edouard Belin
31400 Toulouse Cedex 9

Institut de Recherche en Informatique de Toulouse
Cr Rose Dieng-Kuntz
31400 Toulouse

LAAS-CNRS
7 Avenue du Colonel Roche
31400 Toulouse

Artificial and Natural Intelligence Toulouse Institute
3 Rue Tarfaya
31400 Toulouse

Amélioration de Méthodes de Benders pour un Problème de Planification Robuste sous Incertitude

Mathis FRANCINE-HABAS

Encadrants : Christian Artigues (Directeur de Recherche au LAAS-CNRS)
Romain Guillaume (Enseignant-Chercheur à l'IRIT/ANITI)

Tuteur Universitaire : Mohammed Sbihi (Enseignant-Chercheur à l'ENAC)

Soutenu le : 31 / 08 / 2026

2025-2026

Remerciements

Je tiens à remercier toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce stage et à la rédaction de ce rapport.

Je remercie tout particulièrement Messieurs Christian Artigues et Romain Guillaume pour la confiance qu'ils m'ont accordée ainsi que pour leur aide tout au long de ce projet de recherche.

Je souhaite également remercier l'ensemble de mes professeurs pour la qualité de leur enseignement, qui m'ont permis d'acquérir les connaissances nécessaires à la réussite de ma formation et de ce stage.

Mathis FRANCINE-HABAS

Table des matières

I Contexte de la recherche 2
II Présentation du travail effectué 3
II.I Objectifs 3
II.II Propositions 3
II.III Résultats 3
III Conclusion 4
III.I Synthèse sur le travail effectué 4
III.II Apports professionnels 4
III.III Apports techniques 4
IV Annexes 5

A Méthodologie et Implémentation **6**
I Modélisation Mathématique 6
II Pseudo-code de l'Algorithme 6
III Implémentation en C++ 7

Table des figures

Liste des tableaux

List of Acronyms

Introduction

I Contexte de la recherche

Ce stage s'inscrit au cœur d'un écosystème scientifique toulousain riche, structuré autour d'une collaboration entre trois acteurs majeurs de la recherche en intelligence artificielle et en informatique : l'institut ANITI, l'IRIT et le LAAS-CNRS.

L'impulsion de cette dynamique est notamment portée par ANITI (Artificial and Natural Intelligence Toulouse Institute), un cluster labellisé par la Communauté d'universités et d'établissements de Toulouse. Son ambition est de concevoir des systèmes d'Intelligence Artificielle dont les prises de décision sont explicables, équitables, fiables et robustes, une condition indispensable pour permettre leur déploiement dans des secteurs critiques à haut risque.

Cette volonté de développer des systèmes décisionnels de confiance s'appuie sur l'expertise de deux laboratoires. D'une part, l'Institut de Recherche en Informatique de Toulouse (IRIT), une unité de recherche publique placée sous la tutelle conjointe du Centre National de la Recherche Scientifique (CNRS) et des universités toulousaines. D'autre part, le Laboratoire d'Analyse et d'Architecture des Systèmes (LAAS) est une unité propre du CNRS. Si le LAAS aborde des thématiques très variées, allant de la robotique aux micro-nano-bio-technologies, il se distingue particulièrement par sa recherche théorique et méthodologique sur la conception de lois mathématiques appliquées à la commande et à la décision.

C'est à l'intersection de ces institutions, et plus spécifiquement au sein de l'équipe ROC (Recherche sur des problèmes d'Optimisation Combinatoire) du LAAS, que s'est déroulé mon stage. Positionnée à la frontière entre la Recherche Opérationnelle et l'Intelligence Artificielle, cette équipe offre un cadre scientifique idéal pour aborder les problématiques d'optimisation robuste qui font l'objet de ce rapport.

II Présentation du travail effectué

II.I Objectifs

II.II Propositions

II.III Résultats

III Conclusion

III.I Synthèse sur le travail effectué

III.II Apports professionnels

III.III Apports techniques

IV Annexes

Annexe A

Méthodologie et Implémentation

I Modélisation Mathématique

Voici comment introduire les concepts formels de manière élégante. Les environnements sont numérotés automatiquement en fonction du chapitre.

Définition A.1: Problème Maître Restreint (RMP)

Dans le cadre de la décomposition de Benders, le Problème Maître Restreint contient un sous-ensemble des contraintes du problème original. Il coordonne la recherche en proposant une solution partielle qui sera évaluée par le sous-problème.

Théorème A.1: Convergence de la décomposition

Si le domaine des solutions réalisables est borné et non vide, l'algorithme converge en un nombre fini d'itérations vers l'optimum global du problème de planification.

> ***Preuve.*** À chaque itération, soit la solution actuelle est optimale, soit une nouvelle coupe (d'optimalité ou de faisabilité) est ajoutée au problème maître. L'espace des coupes possibles étant fini pour un problème linéaire en nombres entiers, l'algorithme ne peut cycler indéfiniment. ■

Propriété A.1: Structure des coupes

Les coupes générées agissent comme des minorants stricts de la fonction objectif dans la région de l'espace explorée.

II Pseudo-code de l'Algorithme

L'environnement `algorithm2e` est le standard pour les articles scientifiques. Il génère des blocs clairs avec des lignes de structure.

Algorithme 1 : Décomposition de Benders classique

Données : Modèle MILP initial, Tolérance ε
Résultat : Solution optimale x^*, y^*
Initialiser le RMP sans coupes;
$UB \leftarrow +\infty$;
$LB \leftarrow -\infty$;
while $(UB - LB) > \varepsilon$ **do**
  Résoudre le RMP pour obtenir $\hat{y}$;
  Mettre à jour LB;
  Résoudre le sous-problème (SP) avec $\hat{y}$ fixé;
  if *SP est non-réalisable* **then**
    Générer une coupe de faisabilité (rayon extrême);
    Ajouter la coupe au RMP;
  else
    Mettre à jour UB;
    Générer une coupe d'optimalité (point extrême);
    Ajouter la coupe au RMP;
return *Solution courante*

III Implémentation en C++

Pour l'implémentation de nos heuristiques, nous utilisons les fonctionnalités modernes du langage. Le code est indenté et coloré de manière sobre.

Listing A.1 – Ajout d'une coupe d'optimalité au solveur

```cpp
#include <iostream>
#include <vector>

// Fonction permettant d'ajouter une coupe au modele
void addOptimalityCut(Model& masterProblem, const DualSolution& pi,
    double objVal) {
    Expr cutExpression;

    // Construction de la coupe basique : theta >= pi^T * (b - B*y)
    for (size_t i = 0; i < pi.size(); ++i) {
        cutExpression += pi[i] * masterProblem.getSlack(i);
    }

    // Ajout effectif de la contrainte au probleme maitre
    masterProblem.addConstraint( masterProblem.getTheta() >=
        cutExpression );

    std::cout << "Coupe ajoutee avec succes." << std::endl;
}
```